Évaluation comparative des algorithmes d'équilibrage de charge

Glouton vs Recherche Locale vs Génétique

Mohamed Slimane, Majd Bougatef, Aziz Ouergli

March 3, 2026

1 Objectif

Ce rapport compare trois algorithmes d'équilibrage de charge sur un jeu de données synthétique de planification : **Glouton (LPT)**, **Recherche Locale**, et **Algorithme Génétique**. Nous évaluons :

- le temps d'exécution,
- la précision (taux de correspondance exacte avec l'optimum),
- le taux d'erreur (erreur relative moyenne).

Les graphiques de ce rapport proviennent du pipeline de benchmarking exécuté et dérivé de `load_balancing_benchmark.ipynb`.

2 Configuration expérimentale

- Nombre d'échantillons générés : 1000
- Nombre de tâches par échantillon : 12
- Nombre de serveurs : 3
- Taille du sous-ensemble de test : 200 (stratifié par quantiles de difficulté)
- Graine aléatoire : 42

Statistiques du jeu de données

Métrique	Valeur
Échantillons	1000.0
Tâches par échantillon	12.0
Serveurs	3.0
Charge totale moyenne	552.741
Écart-type de la charge totale	93.855
Difficulté moyenne	1.0336

Table 1: Propriétés globales du jeu de données.

Répartition du sous-ensemble de test

Classe de difficulté	Effectif
0	40
1	40
2	40
3	40
4	40

Table 2: Sous-ensemble de test stratifié (équilibré entre les niveaux de difficulté).

3 Exploration visuelle

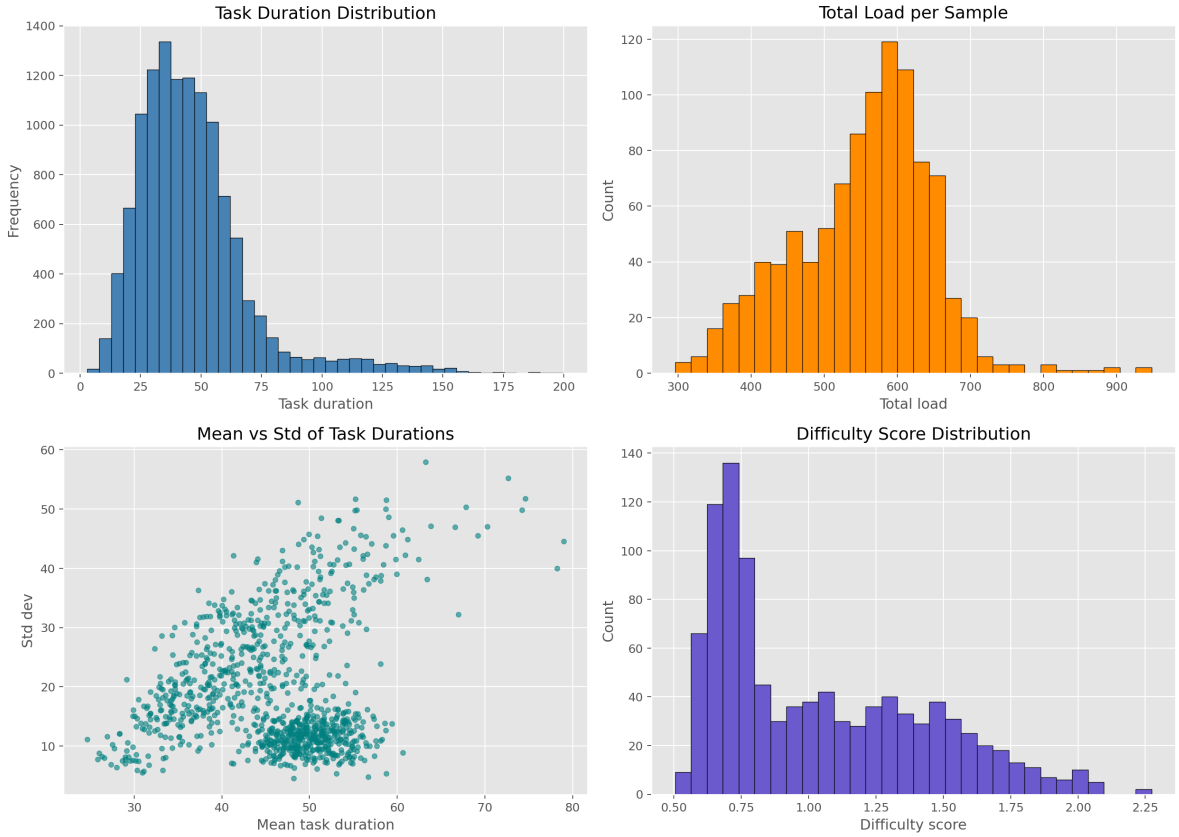

Figure 1: Aperçu du jeu de données : distribution des durées de tâches, charge totale, structure de variance, et score de difficulté.

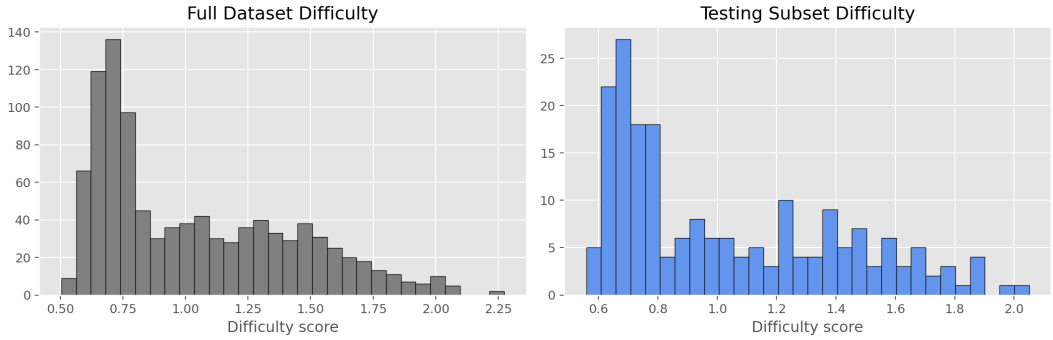

Figure 2: Distribution de la difficulté dans le jeu complet vs le sous-ensemble de test stratifié.

4 Résultats du benchmark des algorithmes

Algorithme	Temps moyen (ms)	Temps médian (ms)	Précision (%)	Taux d'erreur (%)	Pire erreur
Génétique	37.9103	36.0120	26.0	0.7998	3.5
Glouton (LPT)	0.0162	0.0145	9.0	2.1878	8.4
Recherche Locale	0.2418	0.2269	38.0	0.6299	4.3

Table 3: Résumé des performances sur 200 instances de test.

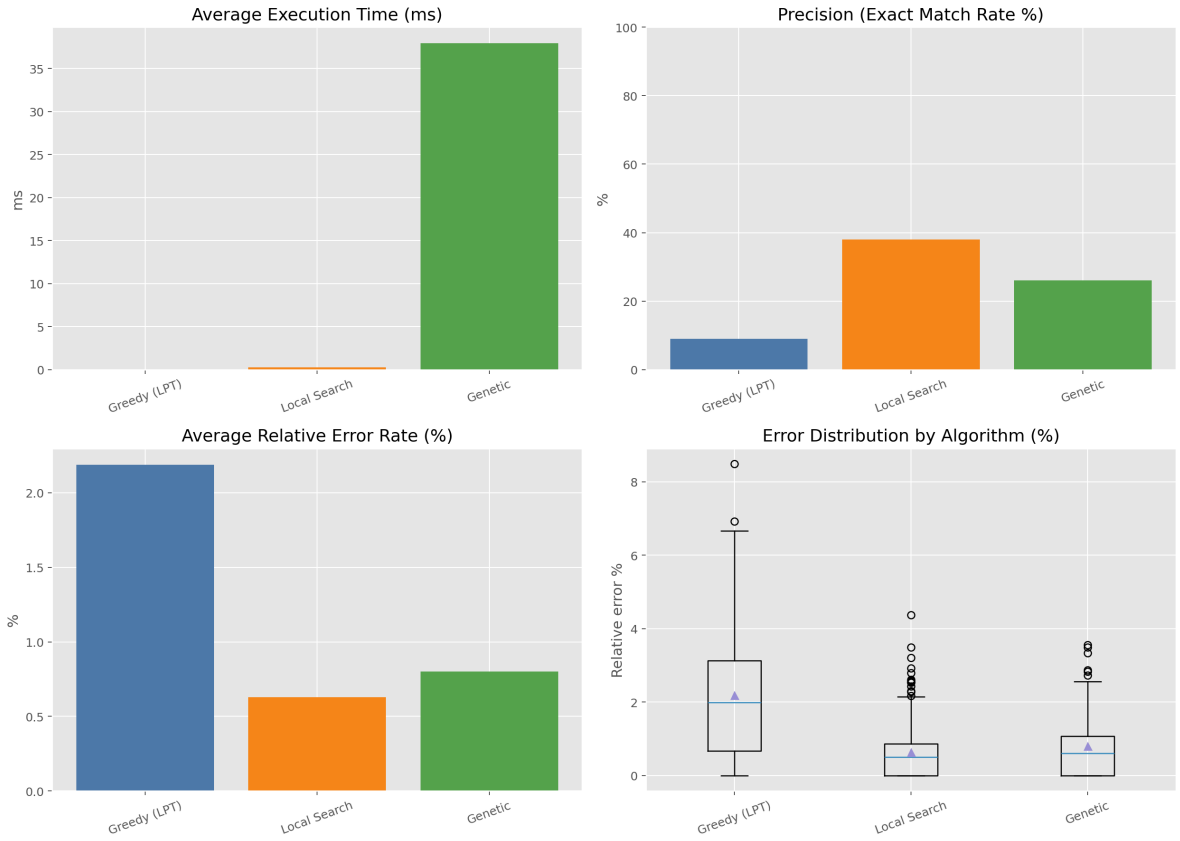

Figure 3: Graphiques de comparaison : temps d'exécution, précision, erreur moyenne, et distribution des erreurs.

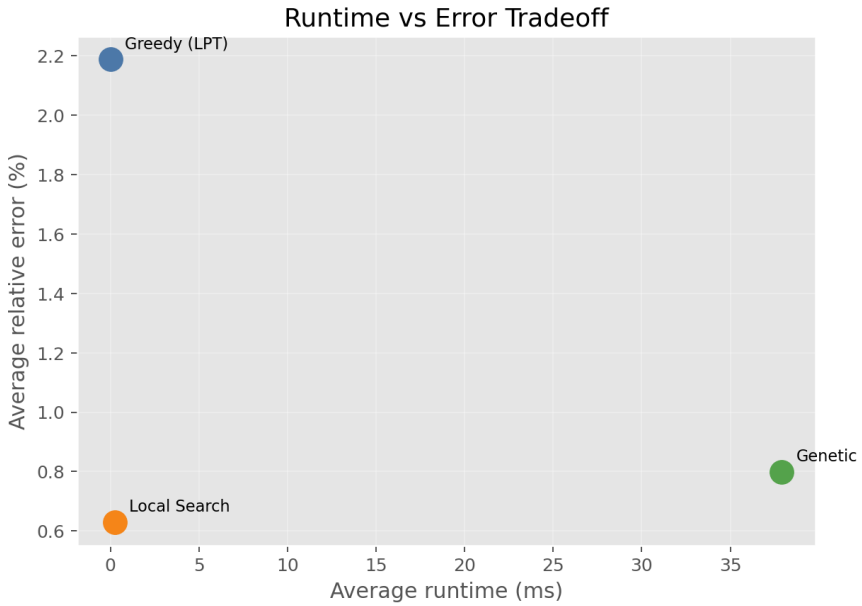

Figure 4: Compromis temps d'exécution vs erreur pour les trois algorithmes.

5 Extraits de code (minimaux)

Extrait 1 : Baseline glouton (LPT)

```
def greedy_lpt(tasks, m=3):
    order = np.argsort(tasks)[::-1]
    loads = np.zeros(m, dtype=int)
    assignment = np.zeros(len(tasks), dtype=int)
    for i in order:
        s = int(np.argmin(loads))
        loads[s] += int(tasks[i])
        assignment[i] = s
    return int(loads.max()), assignment
```

Extrait 2 : Extraction des métriques

```
results['abs_error'] = results['predicted'] - results['optimal']
results['rel_error'] = results['abs_error'] / results['optimal']
results['is_exact'] = (results['predicted'] == results['optimal']).astype(
    int)

summary = results.groupby('algorithm', as_index=False).agg(
    avg_runtime_ms=('runtime_ms', 'mean'),
    precision=('is_exact', 'mean'),
    error_rate=('rel_error', 'mean')
)
```

6 Interprétation

- **Glouton (LPT)** est de loin la méthode la plus rapide, mais elle présente la plus faible précision et la plus grande erreur moyenne.
- **Recherche Locale** fournit la meilleure qualité globale dans cette configuration : précision la plus élevée et erreur moyenne la plus faible, avec un temps d'exécution modéré.
- **Génétique** améliore la qualité par rapport au glouton, mais il est nettement plus lent que les deux alternatives avec les hyperparamètres choisis.
- Le graphique temps d'exécution–erreur confirme un compromis clair : temps quasi instantané (Glouton) vs meilleure qualité d'affectation (Recherche Locale / Génétique).

7 Conclusion finale

Pour cette configuration de benchmark, la **Recherche Locale** offre le meilleur compromis entre vitesse et qualité de solution. Utilisez **Glouton (LPT)** pour des contraintes strictes en temps réel, et **Génétique** lorsque l'on souhaite explorer davantage la qualité des solutions et que l'on peut accepter une latence plus élevée.

A Exemples de lignes d'exécution

Échantillon	Algorithme	Optimal	Prédit	Temps (ms)	Erreur abs	Erreur rel (%)
5	Glouton (LPT)	132	140	0.0185	8	6.0606
5	Recherche Locale	132	135	0.3619	3	2.2727
5	Génétique	132	134	42.1506	2	1.5152
7	Glouton (LPT)	195	200	0.0345	5	2.5641
7	Recherche Locale	195	195	0.4051	0	0.0000
7	Génétique	195	196	43.2046	1	0.5128

Table 4: Lignes d'exécution représentatives issues de la sortie du benchmark.